

Reviewer Pack — Minimal Rank of Geometric Vector Fields and Communication Co...

Entry 9051afe8910a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 23:32:01 UTC

Minimal Rank of Geometric Vector Fields and Communication Complexity Rank Inequality

Entry ID: 9051afe8910a **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 23:32:01 UTC

1. Conjecture

Field A (mathematical branch): Differential Geometry (Geometric Vector Fields) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every d -dimensional geometric vector field on the manifold M , the minimal rank of its holonomy representation $H(M)$ is linearly correlated with its communication complexity rank, such that $\text{rank}(H(M)) = \Theta(\text{rank_comm}(M))$.

Rationale (proposer's reasoning):

Geometric vector fields provide a rich algebraic-geometric structure that could potentially capture the complexity of distributed computations in communication complexity. The minimal rank of the holonomy representation might serve as an invariant that reveals underlying complexity properties.

Taxonomy category: INNOVATIVE_VECTOR_FIELDS (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f51d61073b9b36f2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between the minimal rank of holonomy representations and communication complexity ranks exceeds 0.7 for at least 95% of bootstrap samples with 100 seeds, and all computed communication complexity ranks are within the range $[1, d]$. It is falsified if this condition is not met.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "geometric vector fields" AND "communication complexity rank" - "holonomy representation" AND "matrix rank inequality" - "differential geometry" AND "communication complexity"

Top relevant hits considered: - [s2:10.1038/s41598-023-36865-z] Dual-functional metalenses for the polarization-controlled generation of focalized vector beams in the telecom infrared - [s2:10.1017/S0963548308009656] Learning Complexity vs. Communication Complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Define a simple geometric vector field on a d-dimensional manifold
    def generate_vector_field(d):
        return [[random.uniform(-1, 1) for _ in range(d)] for _ in range(d)]

    # Compute the minimal rank of the holonomy representation
    def min_rank(matrix):
        n = len(matrix)
        if n == 0:
            return 0
        rank = 0
        for i in range(n):
            pivot = matrix[i][i]
            if pivot != 0:
                rank += 1
                for j in range(i + 1, n):
                    factor = matrix[j][i] / pivot
                    for k in range(i, n):
                        matrix[j][k] -= factor * matrix[i][k]
        return rank

    # Compute the communication complexity rank
    def comm_complexity_rank(matrix):
        n = len(matrix)
        if n == 0:
```

```

        return 0
    rank = 0
    for i in range(n):
        pivot = matrix[i][i]
        if pivot != 0:
            rank += 1
            for j in range(i + 1, n):
                factor = matrix[j][i] / pivot
                for k in range(i, n):
                    matrix[j][k] -= factor * matrix[i][k]
    return rank

# Generate random geometric vector fields on d-dimensional manifolds
d = random.randint(2, 5)
vector_fields = [generate_vector_field(d) for _ in range(30)]

# Compute the minimal rank of the holonomy representation and communication complexity rank
ranks_holonomy = [min_rank(field) for field in vector_fields]
ranks_comm = [comm_complexity_rank(field) for field in vector_fields]

# Check if all computed communication complexity ranks are within the range [1, d]
if any(rank < 1 or rank > d for rank in ranks_comm):
    return {
        "metric_name": "communication_complexity_rank",
        "metric_value": None,
        "instances_tested": len(vector_fields),
        "n_max": d,
        "conjecture_holds": False,
        "counterexample": "computed_rank_outside_range"
    }

# Compute the Pearson correlation coefficient
def pearsonr(x, y):
    n = len(x)
    mean_x = sum(x) / n
    mean_y = sum(y) / n
    cov_xy = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(n)) / n
    std_x = math.sqrt(sum((x[i] - mean_x) ** 2 for i in range(n)) / n)
    std_y = math.sqrt(sum((y[i] - mean_y) ** 2 for i in range(n)) / n)
    return cov_xy / (std_x * std_y)

correlation_coefficient = pearsonr(ranks_holonomy, ranks_comm)

# Check if the Pearson correlation coefficient exceeds 0.7
if correlation_coefficient < 0.7:
    return {
        "metric_name": "communication_complexity_rank",
        "metric_value": correlation_coefficient,
        "instances_tested": len(vector_fields),
        "n_max": d,
        "conjecture_holds": False,
        "counterexample": "correlation_coefficient_too_low"
    }

return {
    "metric_name": "communication_complexity_rank",

```

```

        "metric_value": correlation_coefficient,
        "instances_tested": len(vector_fields),
        "n_max": d,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [random.randint(2, 97) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(result["metric_value"] for result in results if result["metric_value"] is not None)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results if result["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.95:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient_too_low\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44c18282.py", line 113, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44c18282.py", line 86, in run_trial
    correlation_coefficient = pearsonr(ranks_holonomy, ranks_comm)
                              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44c18282.py", line 84, in pearsonr
    return cov_xy / (std_x * std_y)
           ~~~~~^~~~~~
ZeroDivisionError: float division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the Pearson correlation coefficient and bootstrapped support fraction as required by the pre-registered criterion. | next: Re-run the test with proper error handling to ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14382
2	preregistration	ollama_remote	glm4:latest	0	0	9692
3	novelty	ollama_remote	glm4:latest	0	0	8699
4	novelty	ollama_remote	glm4:latest	0	0	10333
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	52807
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10535
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13087
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13506
9	judge	ollama_remote	glm4:latest	0	0	21550

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 154591 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/9051afe8910a.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/9051afe8910a.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/9051afe8910a.tar.gz (if generated)